

PIMPRI CHINCHWAD EDUCATION TRUST'S

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

NBA Accredited | NAAC Accredited with 'A' Grade | An Autonomous Institute
AICTE Approved | Permanently Affiliated to SPPU, Pune

Department of Information Technology

Academic Year: 2026-2027

Assignment No. 1: Merge Sort

Name	:	**Om Rajendra Bharambe**
PRN	:	124B1F001
Branch	:	Information Technology (IT)
Semester	:	V
Batch	:	A1
Course Code	:	BIT25PC04
Course Name	:	**Design & Analysis of Algorithms Laboratory**
Guided By	:	Dr. Rajesh N. Phursule
GitHub Repository	:	om29dev/DAA_Lab_Submission
Assignment Link	:	Assignment 1 on GitHub

Assignment No. 1: Merge Sort

1. Problem Statement:

Design and implement a sorting algorithm using Merge Sort to efficiently arrange customer orders based on their timestamps. The solution should handle a large dataset (up to 1 million orders) with minimal computational overhead. Additionally, analyze the time complexity and compare it with traditional sorting techniques.

2. Course Objective:

2.1. To know the basics of computational complexity of various algorithms.
2.2. To select appropriate algorithm design strategies to solve real-world problems.

3. Course Outcome:

3.1. Analyze the asymptotic performance of algorithms.
3.2. Solve computational problems by applying suitable paradigms of Divide and Conquer or Greedy methodologies.

4. Theory:

Merge sort is a popular sorting algorithm known for its efficiency and stability. It follows the divide and conquer approach. It works by recursively dividing the input array into two halves, recursively sorting the two halves and finally merging them back together to obtain the sorted array.

Working of Merge Sort:

- **Divide:** Keep dividing the list or array into two equal halves again and again, until each part has only one element left.
- **Conquer:** Each sub array is sorted individually using the merge sort algorithm.
- **Merge:** The sorted sub arrays are merged back together in sorted order. The process continues until all elements from both sub arrays have been merged.

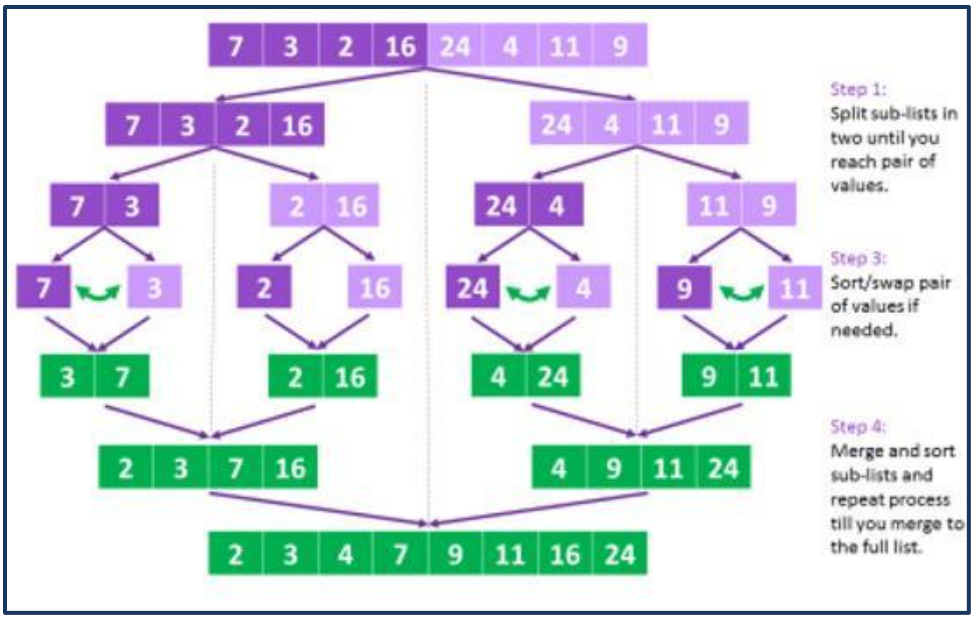


Figure 4.1: Divide and Conquer Execution Tree of Merge Sort

Complexity Analysis of Merge Sort:

- **Divide step:** The array is recursively divided into two halves until each subarray has one element. Number of times we can divide an array of size n = $\log_2(n)$ (height of recursion tree).
- **Merge step:** At each level of recursion, all n elements are merged.
- **Total work:** = number of levels × work per level = $\log_2(n) \times n = O(n \log n)$.

Analysis of Merge Sort Time Complexity:

- **Best Case Time Complexity:** Even if the array is already sorted, Merge Sort still divides and merges every element. Merging sorted sub arrays still requires scanning through all elements. Best Case = O(n log n)
- **Average Case Time Complexity:** For randomly arranged data, the algorithm still divides into halves and merges them. On average, the number of comparisons during merging is proportional to n. Average Case = O(n log n)
- **Worst Case Complexity:** Even if the array is sorted in descending order (worst arrangement for comparisons), Merge Sort still performs the same divide-and-merge procedure. Comparisons may be maximum, but still bounded by O(n log n). Worst Case = O(n log n)

Analysis of Merge Sort Space Complexity:

In merge sort, all elements are copied into an auxiliary array of size N, where N is the number of elements present in the unsorted array. Hence, the space complexity for Merge Sort is O(N).

Merge Sort Algorithm:

- **Step 1:** If it is only one element in the list, consider it already sorted, so return.
- **Step 2:** Divide the list recursively into two halves until it can't no more be divided.
- **Step 3:** Merge the smaller lists into new list in sorted order.

Pseudo code:

```
procedure mergesort( var a as array )
  if ( n == 1 ) return a
  var l1 as array = a[0] ... a[n/2]
  var l2 as array = a[n/2+1] ... a[n]
  l1 = mergesort( l1 )
  l2 = mergesort( l2 )
  return merge( l1, l2 )
end procedure

procedure merge( var a as array, var b as array )
  var c as array
  while ( a and b have elements )
    if ( a[0] > b[0] )
      add b[0] to the end of c
      remove b[0] from b
    else
      add a[0] to the end of c
      remove a[0] from a
    end if
  end while
  while ( a has elements )
    add a[0] to the end of c
    remove a[0] from a
  end while
  while ( b has elements )
    add b[0] to the end of c
    remove b[0] from b
  end while
  return c
end procedure
```

5. Flow Diagram:

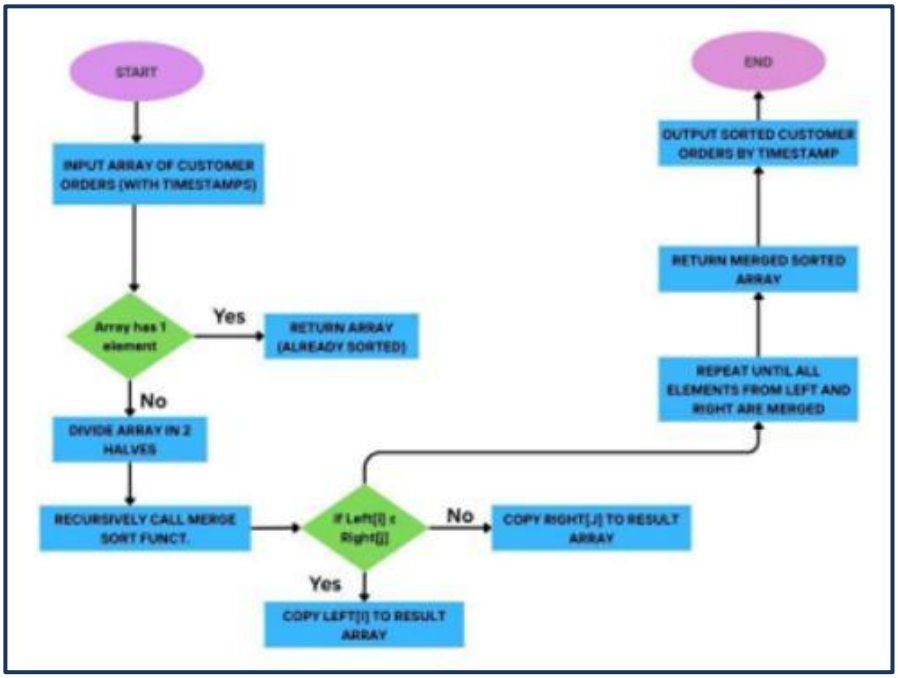


Figure 5.1: Flow Diagram of Customer Order Merge Sort Algorithm

6. Example:

Customer order timestamps: [7, 3, 2, 16, 24, 4, 11, 9]

- **Step 1: Divide**
  - Split into halves → [7, 3, 2, 16] and [24, 4, 11, 9]
  - Further split into pairs → [7, 3], [2, 16] and [24, 4], [11, 9]
  - Split into individual elements → [7], [3], [2], [16], [24], [4], [11], [9]
- **Step 2: Merge & Sort**
  - Merge [7] and [3] → [3, 7]; Merge [2] and [16] → [2, 16]
  - Merge [24] and [4] → [4, 24]; Merge [11] and [9] → [9, 11]
  - Merge [3, 7] and [2, 16] → [2, 3, 7, 16]
  - Merge [4, 24] and [9, 11] → [4, 9, 11, 24]
  - Merge final two halves → [2, 3, 4, 7, 9, 11, 16, 24]

Output:
Orders sorted by timestamp → [2, 3, 4, 7, 9, 11, 16, 24]

7. Code and Output:

Repository: https://github.com/om29dev/DAA_Lab_Submission

Assignment 1 Folder: https://github.com/om29dev/DAA_Lab_Submission/tree/main/Assignment 1

7.1 C++ Implementation Source Code (customer_order_merge_sort.cpp):

```
#include <iostream>
#include <vector>
#include <string>
#include <fstream>
#include <sstream>
#include <iomanip>

using namespace std;

struct Order {
    string orderId;
    string customerId;
    string status;
    string timestamp;
};

void merge(vector<Order>& orders, int low, int mid, int high) {
    vector<Order> left, right;

    for (int i = low; i <= mid; i++)
        left.push_back(orders[i]);

    for (int i = mid + 1; i <= high; i++)
        right.push_back(orders[i]);

    int i = 0, j = 0, k = low;

    while (i < left.size() && j < right.size()) {
        if (left[i].timestamp <= right[j].timestamp)
            orders[k++] = left[i++];
        else
            orders[k++] = right[j++];
    }

    while (i < left.size())
        orders[k++] = left[i++];

    while (j < right.size())
        orders[k++] = right[j++];
}

void mergeSort(vector<Order>& orders, int low, int high) {
    if (low < high) {
        int mid = low + (high - low) / 2;

        mergeSort(orders, low, mid);
        mergeSort(orders, mid + 1, high);
```

```
        merge(orders, low, mid, high);
    }
}

vector<Order> loadOrders(const string& filename) {
    vector<Order> orders;
    ifstream file(filename);

    if (!file.is_open()) {
        cout << "Error opening customer_orders.csv" << endl;
        return orders;
    }

    string line;
    getline(file, line);

    while (getline(file, line)) {
        stringstream ss(line);
        Order o;

        getline(ss, o.orderId, ',');
        getline(ss, o.customerId, ',');
        getline(ss, o.status, ',');
        getline(ss, o.timestamp, ',');

        if (!o.timestamp.empty())
            orders.push_back(o);
    }

    file.close();
    return orders;
}

int main() {
    vector<Order> orders = loadOrders("customer_orders.csv");

    if (orders.empty()) {
        cout << "No valid orders found." << endl;
        return 1;
    }

    cout << "\n============================================\n";
    cout << "       CUSTOMER ORDER SORTING SYSTEM\n";
    cout << "============================================\n";

    cout << "\nSorting orders by purchase timestamp...\n";

    mergeSort(orders, 0, static_cast<int>(orders.size()) - 1);

    cout << "\n============================================\n";
    cout << "        ORDERS SORTED BY TIMESTAMP\n";
    cout << "============================================\n\n";
```

```
    cout << left
         << setw(5) << "No."
         << setw(38) << "Order ID"
         << setw(38) << "Customer ID"
         << setw(16) << "Status"
         << "Purchase Timestamp" << '\n';

    cout << string(120, '-') << '\n';

    int count = 0;

    for (const Order& o : orders) {
        if (count >= 10)
            break;

        cout << left
             << setw(5) << count + 1
             << setw(38) << o.orderId
             << setw(38) << o.customerId
             << setw(16) << o.status
             << o.timestamp << '\n';

        count++;
    }

    cout << "\n================================================\n";
    cout << "Total orders sorted: " << orders.size() << '\n';
    cout << "Orders displayed: " << count << '\n';
    cout << "================================================\n";

    return 0;
}
```

7.2 Execution Output:

Below is the terminal execution output verifying the sorting of customer order records by timestamps:

```
om29dev@DESKTOP-MH6NVIC:~/daa$ vi customer_order_merge_sort.cpp
om29dev@DESKTOP-MH6NVIC:~/daa$ ls
customer_order_merge_sort.cpp  customer_orders.csv
om29dev@DESKTOP-MH6NVIC:~/daa$ g++ customer_order_merge_sort.cpp -o customer_order_merge_sort
om29dev@DESKTOP-MH6NVIC:~/daa$ ./customer_order_merge_sort

============================================
       CUSTOMER ORDER SORTING SYSTEM
============================================

Sorting orders by purchase timestamp...

============================================
        ORDERS SORTED BY TIMESTAMP
============================================

No.  Order ID                            Customer ID                         Status          Purchase Timestamp
-------------------------------------------------------------------------------------------------------------------
1    f2d571f5-b690-4816-a980-6f9f73235e8d  01c9a79b-83cc-462c-b393-d8d65a72be23  processing      2017-01-01 07:34:02
2    2518305a-1e3d-46fb-bb27-ec92ee8efe88  570b0cca-8ba2-48b0-aaa9-d9ccefbdfff5  delivered       2017-01-01 10:39:39
3    5581a9ae-ebea-4d13-a8b9-a39b7d27689f  cc4d5f5f-13d5-421c-93da-1c56a0d9620d  delivered       2017-01-02 07:31:39
4    83803c35-b19e-4921-b5e0-24059dd0a8b8  36ea7585-188a-4516-8fbd-594b4553e22f  delivered       2017-01-02 16:10:44
5    99c9e90f-592a-4b63-949a-bf7cc314e4a3  72a5c5b8-ce9d-4f2a-9804-8c4177f05a04  delivered       2017-01-03 12:29:49
6    c48603e9-a07c-4be0-a12e-e86d2275b873  1d8edbb8-49a6-4281-b3e6-bb596a8b9e23  shipped         2017-01-03 18:05:47
7    eabcc1c0-ca92-41d5-976d-7cf5e53508a0  4ba0e2a7-077b-4ed8-a717-2b51c8ad702f  delivered       2017-01-05 07:21:46
8    bdccc549-ef67-449a-bf98-6ecd6f5ef1e6  4b5fa310-a3ff-4ffa-8a6e-1a1a829ebabe  delivered       2017-01-05 13:54:39
9    a78dd61e-8e7e-460c-8b35-f444fa4f7c1f  8ac710a4-5d1e-489c-baf4-cac5b02a26c3  delivered       2017-01-06 01:08:44
10   7f51d53e-2f2a-46c3-b1e0-58255a500111  c5655ae4-dd04-4900-b0ba-f693d5e256f8  delivered       2017-01-06 14:02:49

============================================
Total orders sorted: 956
Orders displayed: 10
============================================
om29dev@DESKTOP-MH6NVIC:~/daa$
```

Figure 7.1: Program Execution and Sorted Customer Orders Output

8. Conclusion:

Merge Sort is a highly efficient and stable sorting algorithm that organizes customer orders based on their timestamps with O(n log n) complexity. Compared to traditional algorithms like Bubble Sort or Insertion Sort, which are too slow for large inputs, Merge Sort can easily handle up to 1 million records without performance issues. Its ability to maintain the correct order and scalability makes it an excellent choice for real-world applications such as e-commerce, order tracking, and scheduling systems.